

AWS 3-Tier Highly Available Web Application

Production-Grade Project Documentation

Created By: Anuj Dhirde

Date: 18 September 2026

A multi-tier e-commerce application demonstrating AWS networking, load balancing, multi-AZ compute, managed PostgreSQL, S3 deployment, IAM, private connectivity, and HTTPS.

1. Architecture Overview

The project implements a traditional three-tier architecture consisting of a Web/Presentation Layer, Application Layer, and Database Layer. The architecture diagram shows the client path through Route 53, ACM and the Application Load Balancer before requests reach EC2 application servers and RDS PostgreSQL.

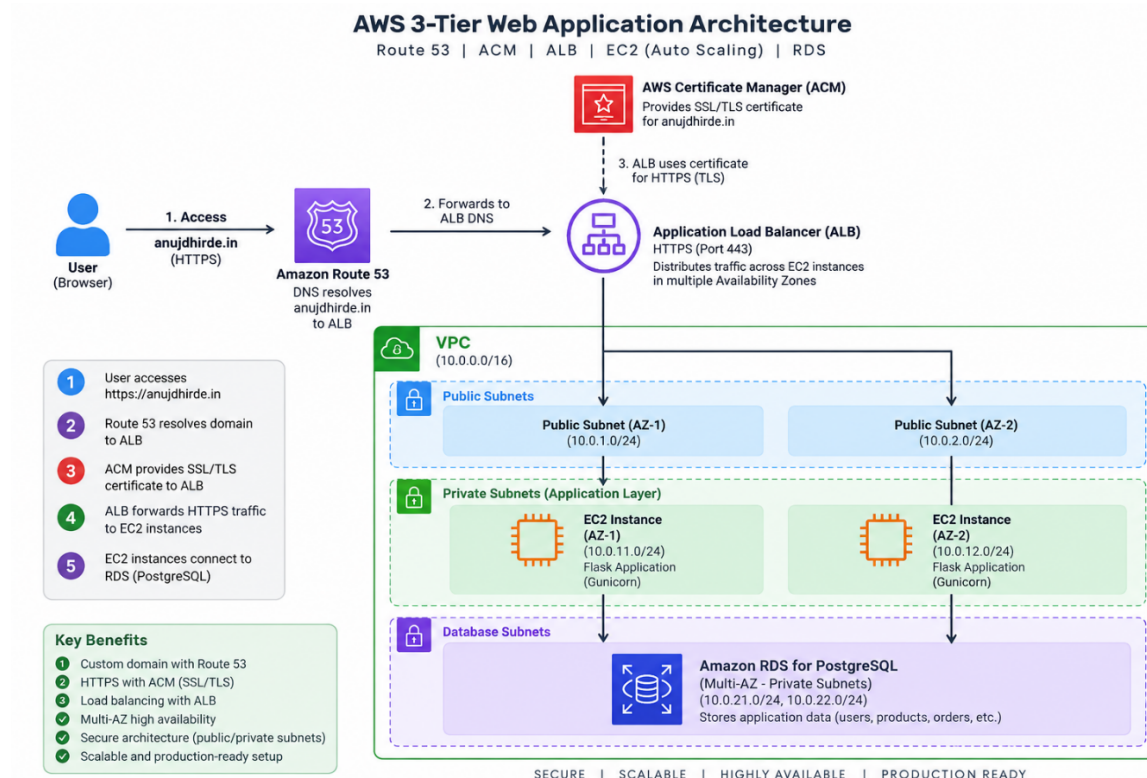


Figure 1 — AWS three-tier architecture overview.

2. Component Breakdown

AWS Component	Role
Amazon VPC	Isolated network boundary for the application
Public Subnets	Host the internet-facing ALB
Private Application Subnets	Host EC2 application servers
Private Database Subnets	Provide network placement for the database layer
Internet Gateway	Internet connectivity for public subnet routing
Route Tables	Control routing for presentation, application, and database tiers
Application Load Balancer	Receives client traffic and distributes requests to EC2
Target Group	Registers EC2 instances as ALB targets
Amazon EC2	Runs the Flask application
NAT Gateway	Temporary outbound connectivity for private EC2
Amazon S3	Stores the application ZIP package
S3 Gateway VPC Endpoint	Private EC2-to-S3 connectivity
IAM Role	Authorizes EC2 access to S3
Amazon RDS for PostgreSQL	Persistent relational database
AWS Certificate Manager	SSL/TLS certificate for HTTPS
Amazon Route 53	DNS resolution for the custom domain

3. Step 1 — Create the VPC and Six Subnets

3.1 Create the VPC

Created a VPC named **application-vpc** to provide the network boundary.

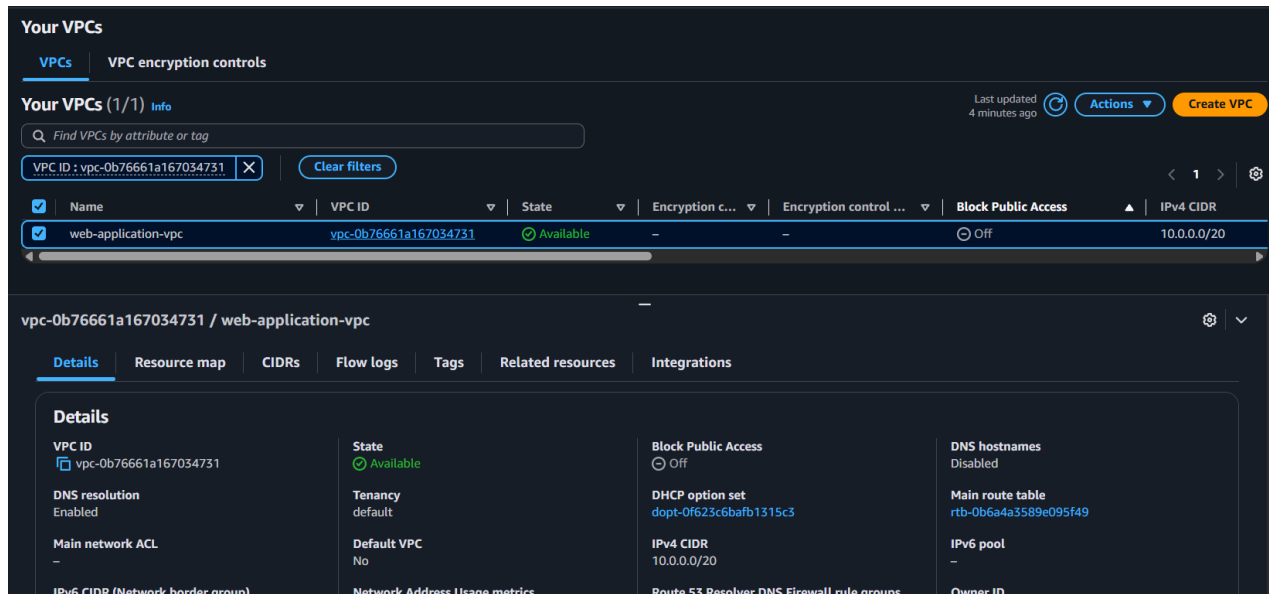


Figure 2 — VPC configuration.

3.2 Create Public Presentation Subnets

Created two public subnets in different Availability Zones for the Application Load Balancer.

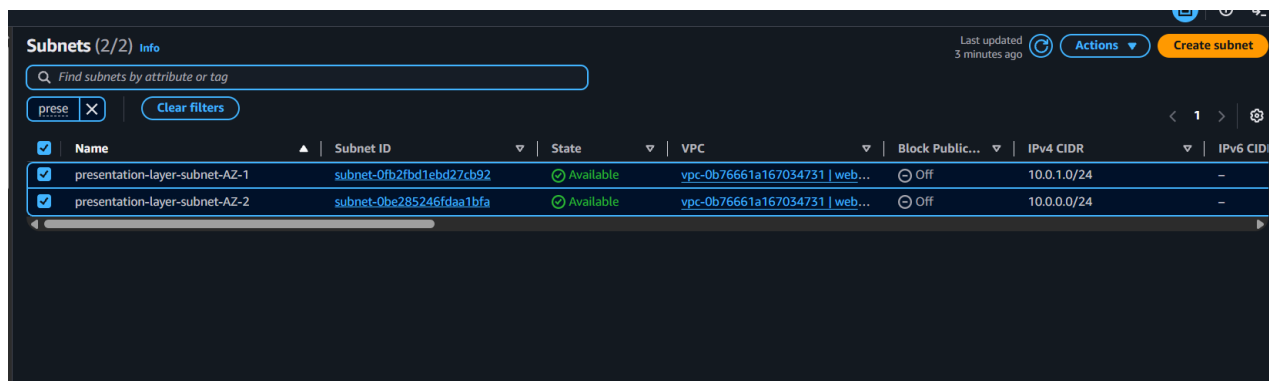


Figure 3 — Public presentation subnets.

3.3 Create Private Application Subnets

Created two private application subnets across the two Availability Zones for EC2.

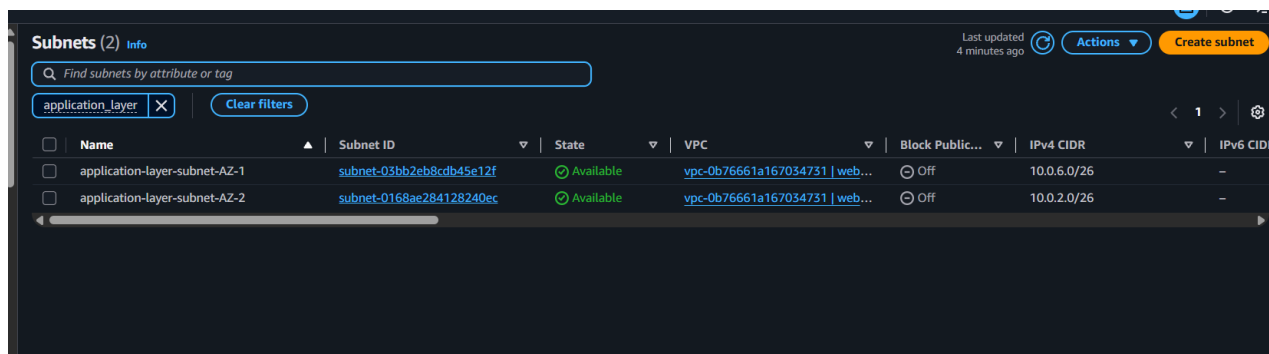


Figure 4 — Private application subnets.

3.4 Create Private Database Subnets

Created two private database subnets in separate Availability Zones for the database layer.

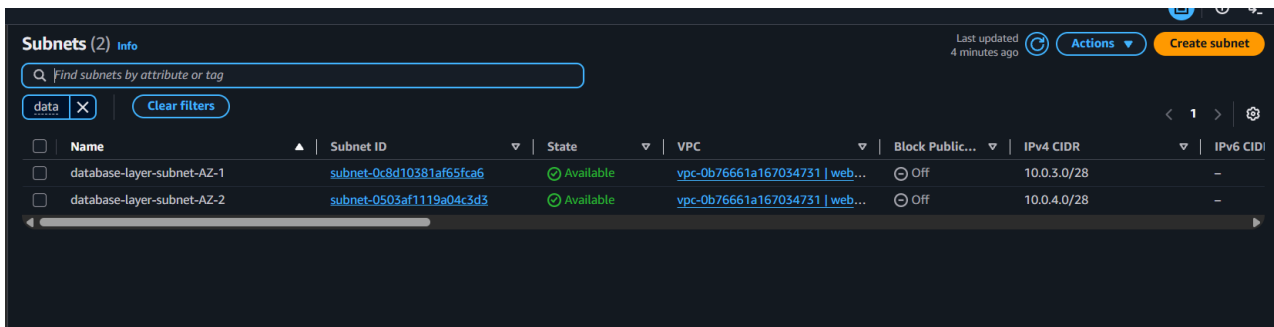


Figure 5 — Database subnets.

3.5 Create and Attach the Internet Gateway

Created an Internet Gateway and attached it to the VPC.

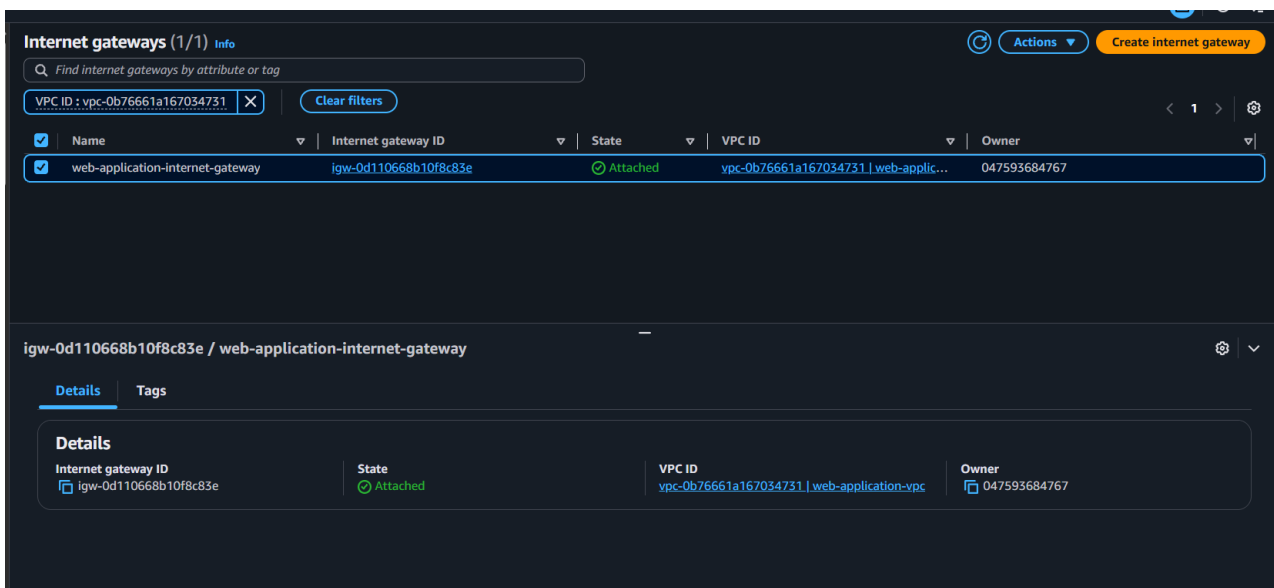


Figure 6 — Internet Gateway.

3.6 Configure Route Tables

Created route tables for the presentation, application, and database layers.

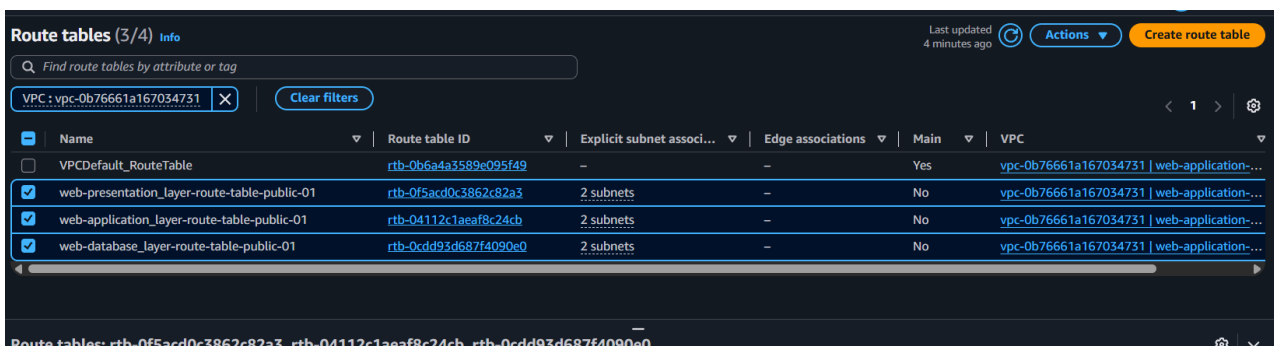


Figure 7 — Route tables.

4. Step 2 — EC2 Instances, Target Group and ALB

4.1 Launch EC2 Instances

Created two EC2 instances in separate application subnets across AZ-1 and AZ-2.

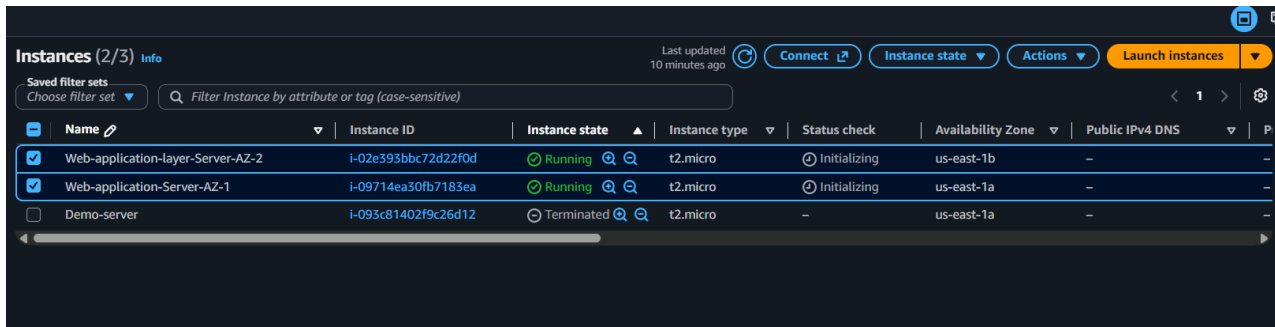


Figure 8 — EC2 instances.

4.2 Create the Target Group

Created **web-application-target-group** and registered both EC2 instances.

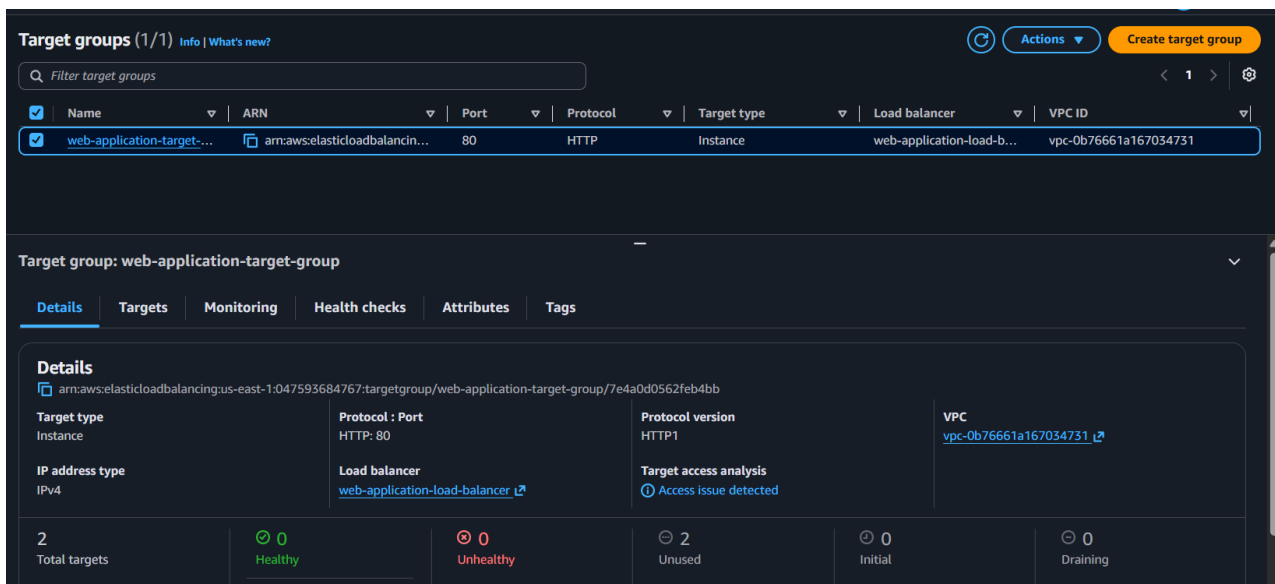


Figure 9 — Target group.

4.3 Create the Application Load Balancer

Created **web-application-load-balancer**, associated it with the two public presentation subnets, and connected the target group.

Load balancers (1/1)

What's new?

Actions

Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

< 1 >

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security groups
☒	web-application-load-b...	Active	application	Internet-facing	IPv4	vpc-0b76661a167034731	2 Availability Zones	sg-0979c7ee676f...

Load balancer: web-application-load-balancer

Details

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Details

Load balancer type

Application

Scheme

Internet-facing

Status

Active

Hosted zone

Z35SXDOTRQ7X7K

VPC

vpc-0b76661a167034731

Availability Zones

subnet-09ed81951d7b2d544 us-east-1b (use1-az2)

subnet-0fb2fbd1ebd27cb92 us-east-1a (use1-az1)

Load balancer IP address type

IPv4

Date created

September 8, 2026, 17:15 (UTC+05:30)

Load balancer ARN

DNS name

Info

Figure 10 — Application Load Balancer.

AWS 3-Tier Highly Available Web Application — Anuj Dhirde

Page 6

5. Step 3 — Temporary NAT Gateway Connectivity

5.1 Provide Outbound Internet Access

Configured NAT Gateway connectivity so private EC2 instances could download Apache and required application dependencies.

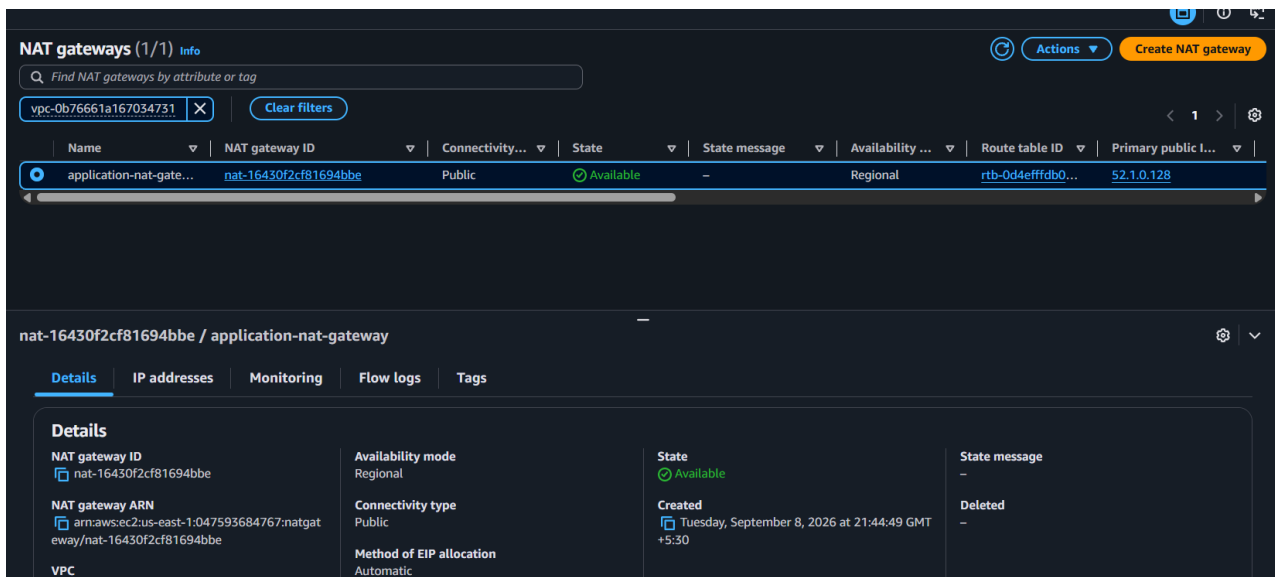


Figure 11 — NAT Gateway and routing.

6. Step 4 — Deploy the Application from Amazon S3

6.1 Upload the Application Package

Created an S3 bucket and uploaded the application ZIP package in US East (N. Virginia).

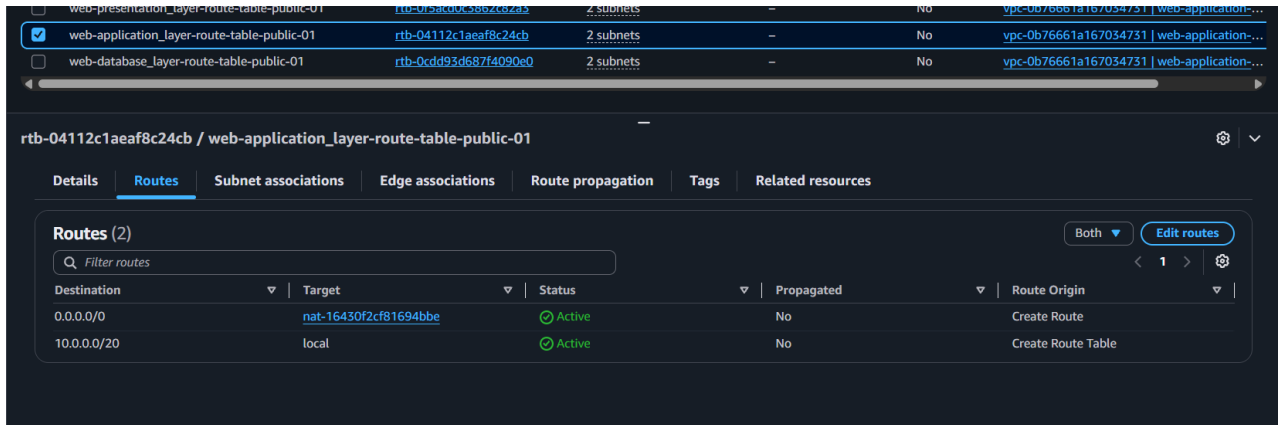


Figure 12 — S3 application package.

6.2 Create an S3 Gateway VPC Endpoint

Created a Gateway VPC Endpoint to allow EC2 to access S3 privately.

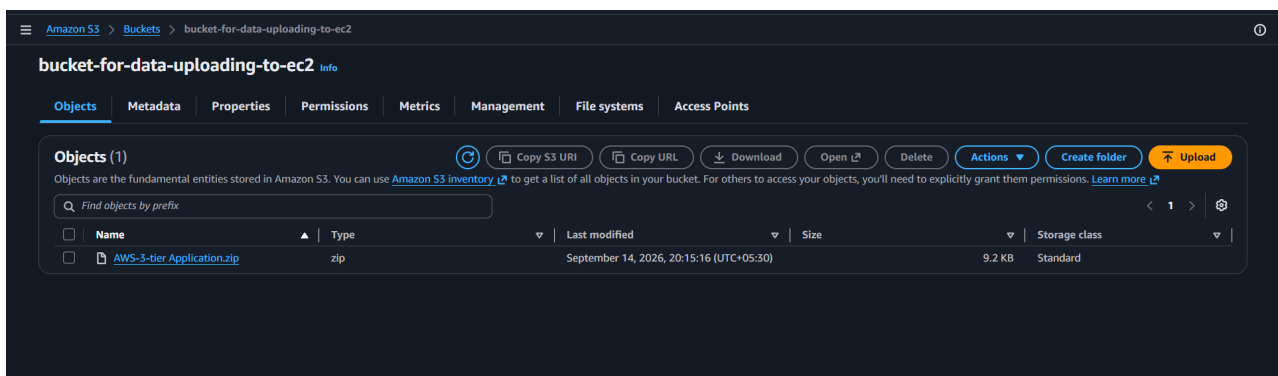


Figure 13 — S3 Gateway VPC Endpoint.

6.3 Attach an IAM Role

Assigned an IAM role to EC2 to authorize access to the S3 bucket.

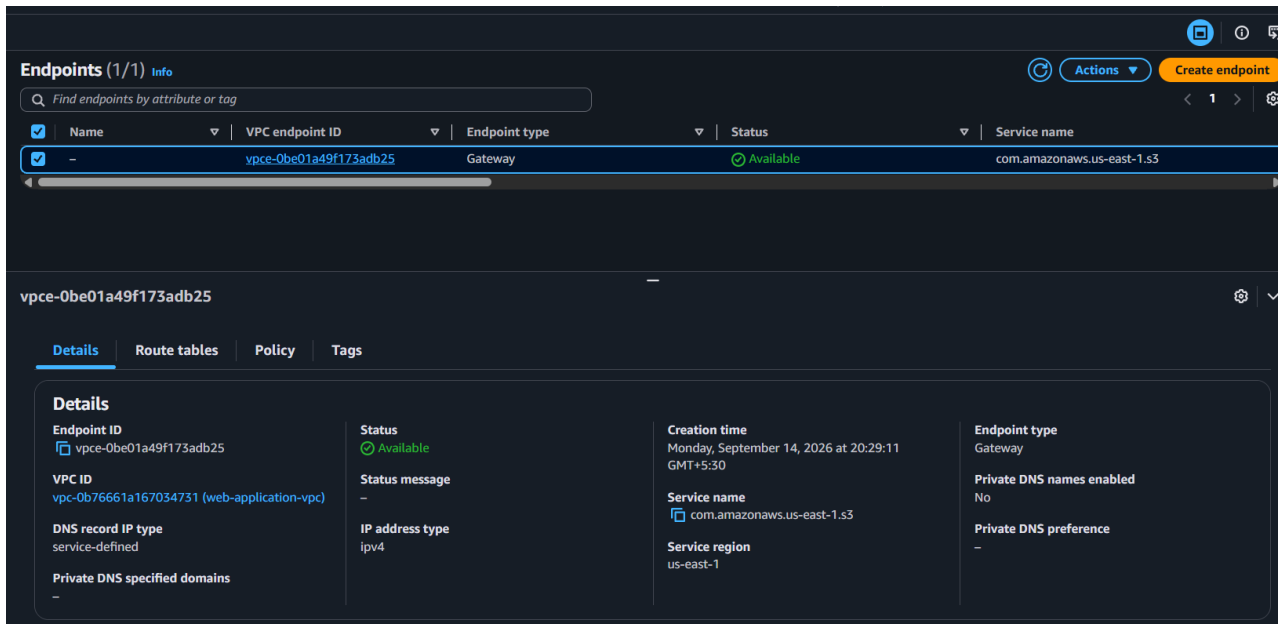


Figure 14 — EC2 IAM role.

6.4 Download the Application

Validated connectivity and permissions, then downloaded the application package from S3.

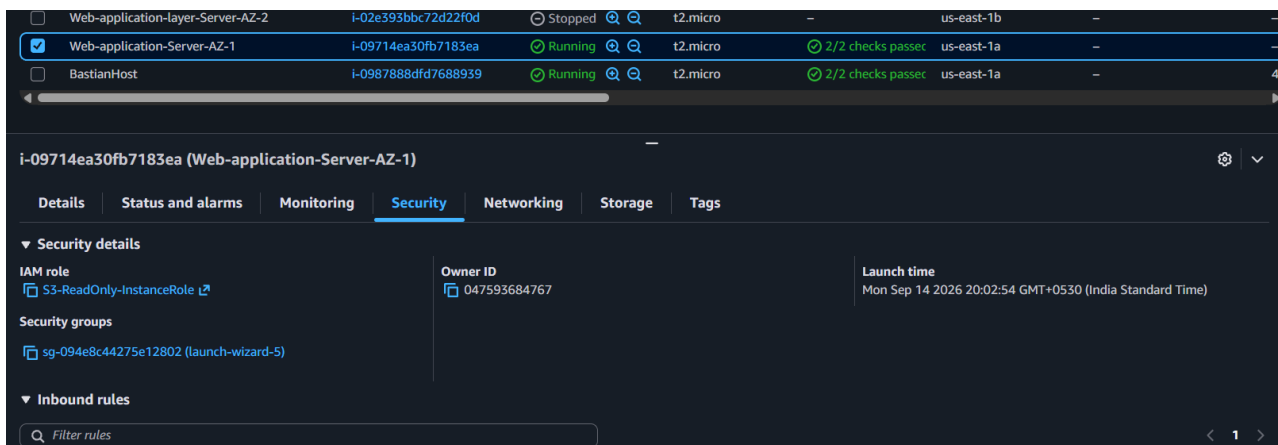


Figure 15 — Application download.

6.5 Extract the Application

Extracted the application package into the application directory.

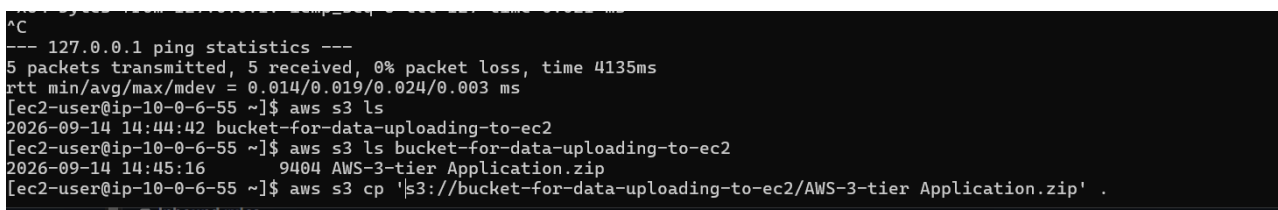


Figure 16 — Application files.

6.6 Create a Python Virtual Environment

Created an isolated Python virtual environment for application dependencies.

```

2026-09-14 14:45:16          9404 AWS-3-tier Application.zip
[ec2-user@ip-10-0-6-55 ~]$ cd /opt/
[ec2-user@ip-10-0-6-55 opt]$ cd ecommerce-website/
[ec2-user@ip-10-0-6-55 ecommerce-website]$ ls
AWS-3-tier Application
[ec2-user@ip-10-0-6-55 ecommerce-website]$ cd 'AWS-3-tier Application'
[ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ ls
README.md app.py requirements.txt static templates venv
[ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ ls-lh
-bash: ls-lh: command not found
[ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ ls -lh
total 16K
-rw-rw-rw-. 1 root    root    2.4K Sep 14 13:44 README.md
-rw-rw-rw-. 1 root    root    6.3K Sep 14 14:29 app.py
-rw-rw-rw-. 1 root    root    16 Sep 14 13:44 requirements.txt
drwxrwxrwx. 2 root    root    23 Sep 14 13:44 static
drwxrwxrwx. 2 root    root   133 Sep 14 13:44 templates
drwxr-xr-x. 5 ec2-user ec2-user 74 Sep 14 17:14 venv
[ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ |

```

Figure 17 — Python virtual environment.

6.7 Install Dependencies

Installed Flask and dependencies listed in **requirements.txt**.

```

-rw-rw-rw-. 1 root    root    6.3K Sep 14 14:29 app.py
-rw-rw-rw-. 1 root    root    16 Sep 14 13:44 requirements.txt
drwxrwxrwx. 2 root    root    23 Sep 14 13:44 static
drwxrwxrwx. 2 root    root   133 Sep 14 13:44 templates
drwxr-xr-x. 5 ec2-user ec2-user 74 Sep 14 17:14 venv
[ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ venv
-bash: venv: command not found
[ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ source venv/bin/activate
(venv) [ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ |

```

Figure 18 — Dependency installation.

6.8 Validate the Application

Started the Flask application and verified successful startup.

```

2.7 kB)
Collecting werkzeug>=3.1.0 (from Flask<4.0,>=3.0->-r requirements.txt (line 1))
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Collecting zipp>=3.20 (from importlib-metadata>=3.6.0->Flask<4.0,>=3.0->-r requirements.txt (line 1))
  Downloading zipp-3.23.1-py3-none-any.whl.metadata (3.6 kB)
  Downloading flask-3.1.3-py3-none-any.whl (103 kB)
  Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
  Downloading click-8.1.8-py3-none-any.whl (98 kB)
  Downloading importlib_metadata-8.7.1-py3-none-any.whl (27 kB)
  Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
  Downloading jinja2-3.1.6-py3-none-any.whl (134 kB)
  Downloading markupsafe-3.0.3-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)
  Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
  Downloading zipp-3.23.1-py3-none-any.whl (10 kB)
Installing collected packages: zipp, markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, importlib-metadata, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.1.8 importlib-metadata-8.7.1 itsdangerous-2.2.0 jinja2-3.1.6 ma
rkupsafe-3.0.3 werkzeug-3.1.8 zipp-3.23.1

```

Figure 19 — Flask application running.

6.9 Replicate the Application Environment

Created a snapshot of the working EC2 environment for use in the second Availability Zone.

```
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.1.8 importlib-metadata-8.7.1 itsdangerous-2.2.0 jinja2-3.1.6 ma
rkupsafe-3.0.3 werkzeug-3.1.8 zipp-3.23.1
(venv) [ec2-user@ip-10-0-6-55 AWS-3-tier Application]$ python app.py
 * Serving Flask app 'app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8080
 * Running on http://10.0.6.55:8080
Press CTRL+C to quit
127.0.0.1 - - [14/Sep/2026 18:12:52] "GET / HTTP/1.1" 200 -
```

Figure 20 — EC2 snapshot.

7. Step 5 — Configure Amazon RDS for PostgreSQL

7.1 Create the PostgreSQL Database

Created the Amazon RDS PostgreSQL database for persistent application data.

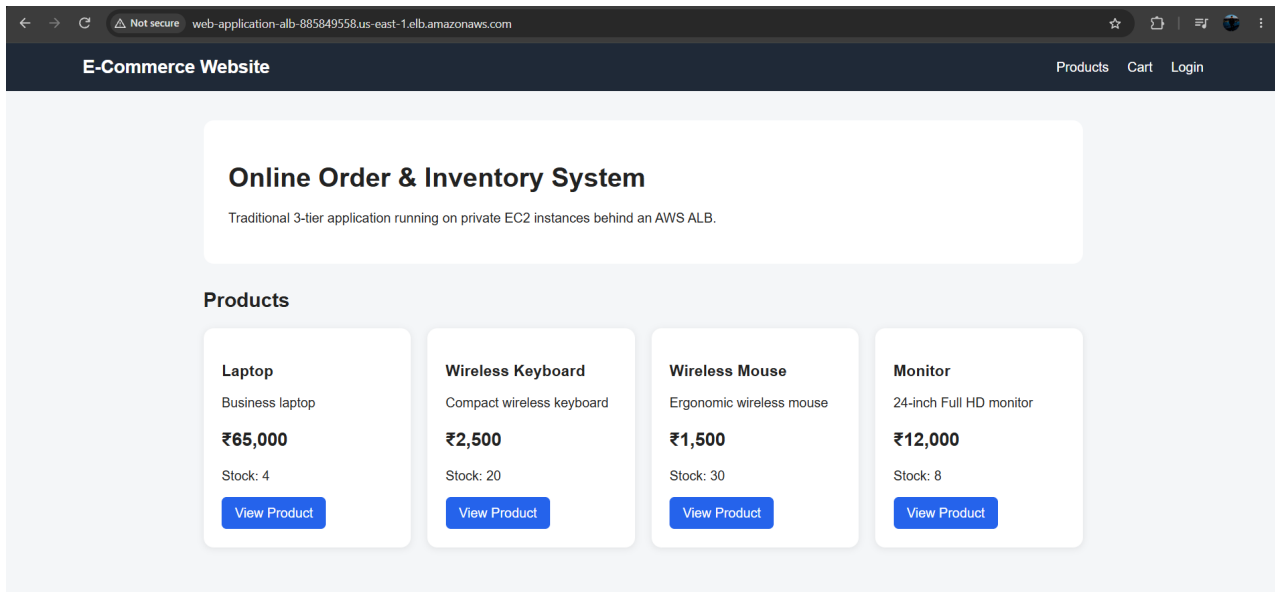


Figure 21 — RDS PostgreSQL.

7.2 Configure Database Connectivity

Configured PostgreSQL access on port 5432 and tested connectivity from EC2.

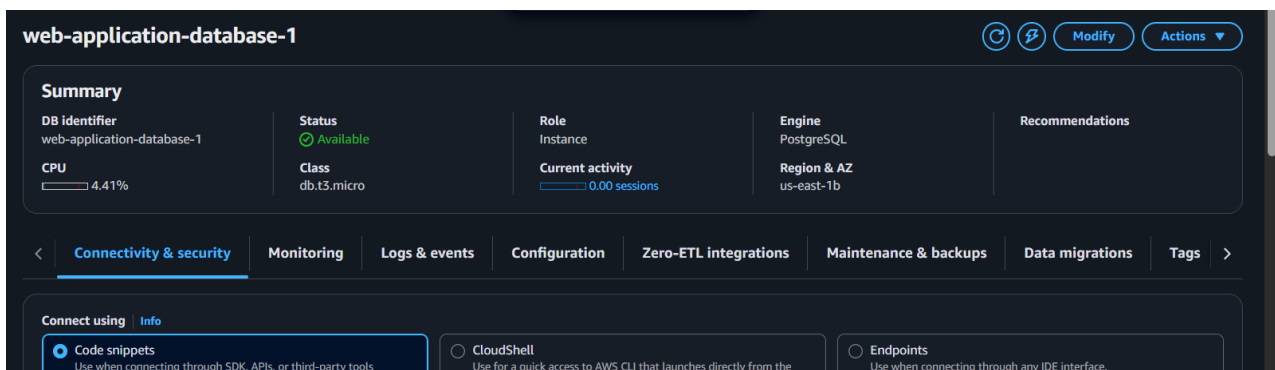


Figure 22 — RDS connectivity.

7.3 Deploy the E-Commerce Database

Created the **ecommerce** database and deployed the required application data structures.

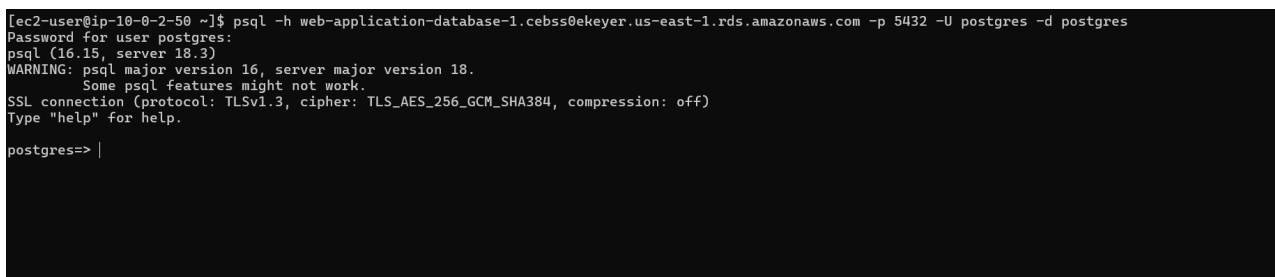


Figure 23 — Database and application data.

8. Step 6 — Validate the Application Servers

8.1 EC2 — AZ-1

Validated the application server in the first Availability Zone.

```
(venv) [root@ip-10-0-2-50 aws-shop]# python -m py_compile app.py
(venv) [root@ip-10-0-2-50 aws-shop]# python -c "from app import app, db, User, Product, Order, OrderItem; app.app_context().push(); print('Users:', User.query.count()); print('Products:', Product.query.count()); print('Orders:', Order.query.count()); print('Order items:', OrderItem.query.count())"
Users: 2
Products: 4
Orders: 0
Order items: 0
(venv) [root@ip-10-0-2-50 aws-shop]# python -c "from app import app, Product; app.app_context().push(); [(print(p.id, p.name, float(p.price), p.stock)) for p in Product.query.order_by(Product.id).all()]"
1 Laptop 65000.0 10
2 Wireless Keyboard 2500.0 20
3 Wireless Mouse 1500.0 30
4 Monitor 12000.0 8
(venv) [root@ip-10-0-2-50 aws-shop]# python -c "from app import app, User; app.app_context().push(); u=User.query.filter_by(email='anuj-admin@example.com').first(); print(u.email, u.role, u.password_hash)"
anuj-admin@example.com admin admin123
(venv) [root@ip-10-0-2-50 aws-shop]# python -m py_compile app.py
(venv) [root@ip-10-0-2-50 aws-shop]# python -c "from app import app, db, User, Product, Order, OrderItem; app.app_context().push(); print('Users:', User.query.count()); print('Products:', Product.query.count()); print('Orders:', Order.query.count()); print('Order items:', OrderItem.query.count())"
Users: 2
Products: 4
Orders: 0
Order items: 0
(venv) [root@ip-10-0-2-50 aws-shop]# python -c "from app import app, Product; app.app_context().push(); [(print(p.id, p.name, float(p.price), p.stock)) for p in Product.query.order_by(Product.id).all()]"
1 Laptop 65000.0 10
2 Wireless Keyboard 2500.0 20
3 Wireless Mouse 1500.0 30
4 Monitor 12000.0 8
```

Figure 24 — Application server in AZ-1.

8.2 EC2 — AZ-2

Validated the application server in the second Availability Zone.

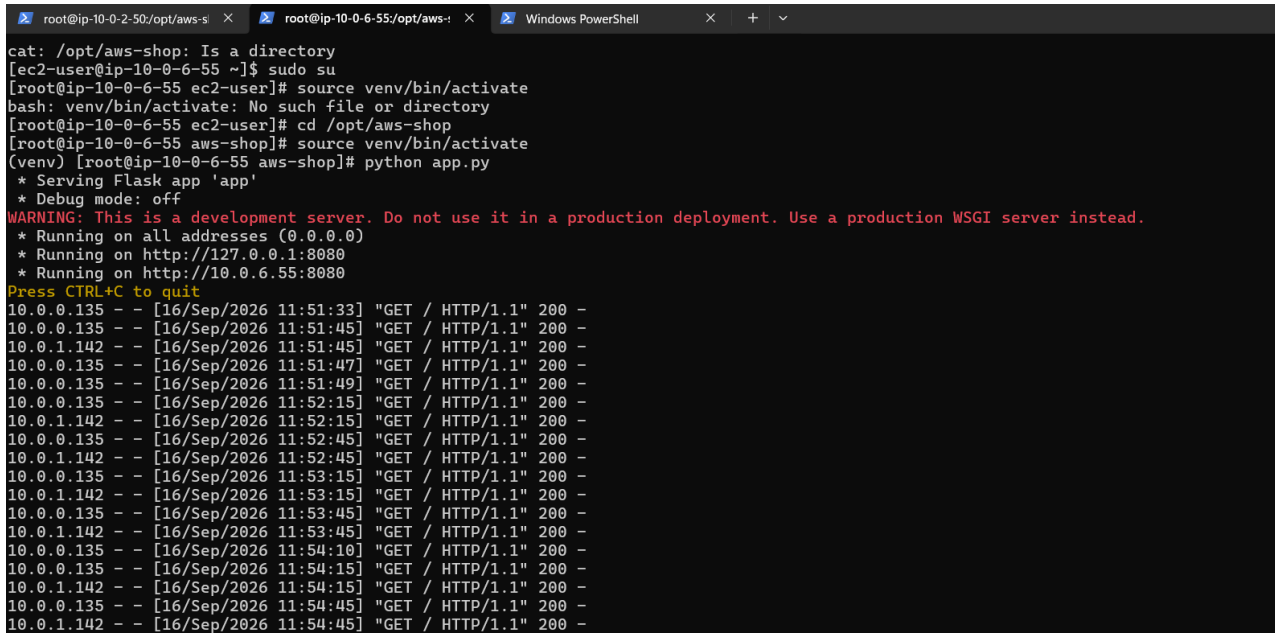
```
root@ip-10-0-2-50/opt/aws-s: x root@ip-10-0-6-55/opt/aws-s x Windows PowerShell x + v
[ec2-user@ip-10-0-2-50 aws-shop]$ sudo su
[root@ip-10-0-2-50 aws-shop]# source venv/bin/activate
(venv) [root@ip-10-0-2-50 aws-shop]# python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.0.2.50:8080
Press CTRL+C to quit
10.0.1.142 - - [16/Sep/2026 11:49:33] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:49:39] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:49:52] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:50:09] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:50:22] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:50:39] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:50:52] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:51:09] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:51:22] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:51:33] "GET /static/style.css HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:51:40] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:51:47] "GET /static/style.css HTTP/1.1" 304 -
10.0.0.135 - - [16/Sep/2026 11:51:50] "GET /static/style.css HTTP/1.1" 304 -
10.0.0.135 - - [16/Sep/2026 11:51:52] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:52:10] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:52:22] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:52:40] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:52:52] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:53:10] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:53:22] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:53:40] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:53:52] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:54:10] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:54:10] "GET /static/style.css HTTP/1.1" 304 -
10.0.0.135 - - [16/Sep/2026 11:54:22] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:54:40] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:54:52] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:55:10] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:55:22] "GET / HTTP/1.1" 200 -
```

Figure 25 — Application server in AZ-2.

9. Step 7 — Configure HTTPS with AWS Certificate Manager

9.1 Provision the ACM Certificate

Generated the SSL/TLS certificate using AWS Certificate Manager.



```
root@ip-10-0-2-50:/opt/aws-si x root@ip-10-0-6-55:/opt/aws-si x Windows PowerShell x + v
cat: /opt/aws-shop: Is a directory
[ec2-user@ip-10-0-6-55 ~]$ sudo su
[root@ip-10-0-6-55 ec2-user]# source venv/bin/activate
bash: venv/bin/activate: No such file or directory
[root@ip-10-0-6-55 ec2-user]# cd /opt/aws-shop
[root@ip-10-0-6-55 aws-shop]# source venv/bin/activate
(venv) [root@ip-10-0-6-55 aws-shop]# python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8080
* Running on http://10.0.6.55:8080
Press CTRL+C to quit
10.0.0.135 - - [16/Sep/2026 11:51:33] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:51:45] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:51:45] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:51:47] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:51:49] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:52:15] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:52:15] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:52:45] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:52:45] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:53:15] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:53:15] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:53:45] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:53:45] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:54:10] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:54:15] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:54:15] "GET / HTTP/1.1" 200 -
10.0.0.135 - - [16/Sep/2026 11:54:45] "GET / HTTP/1.1" 200 -
10.0.1.142 - - [16/Sep/2026 11:54:45] "GET / HTTP/1.1" 200 -
```

Figure 26 — ACM certificate.

9.2 Configure the ALB HTTPS Listener

Added an HTTPS listener on port 443, associated the ACM certificate, and permitted HTTPS traffic.

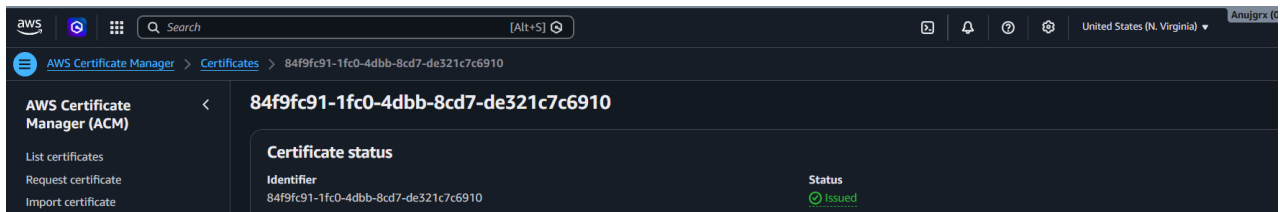


Figure 27 — HTTPS listener.

10. Step 8 — Application Demonstration

10.1 Home Page

The application home page displays the online order and inventory system and its products.

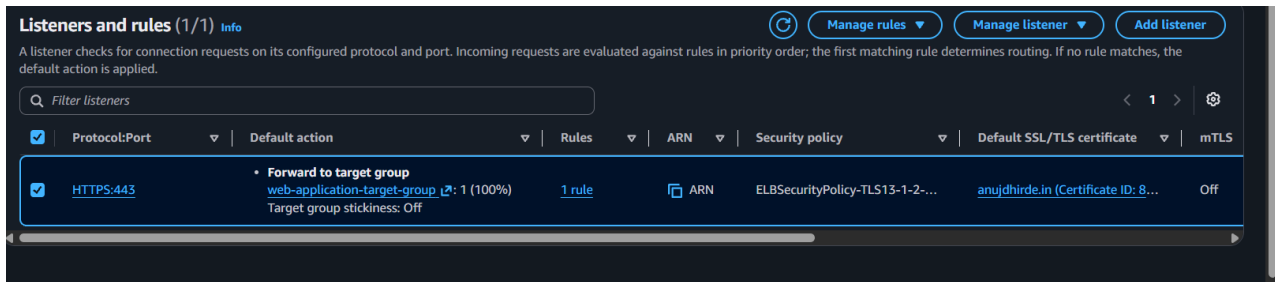


Figure 28 — Home page.

10.2 User Authentication

Users can authenticate through the application login workflow.

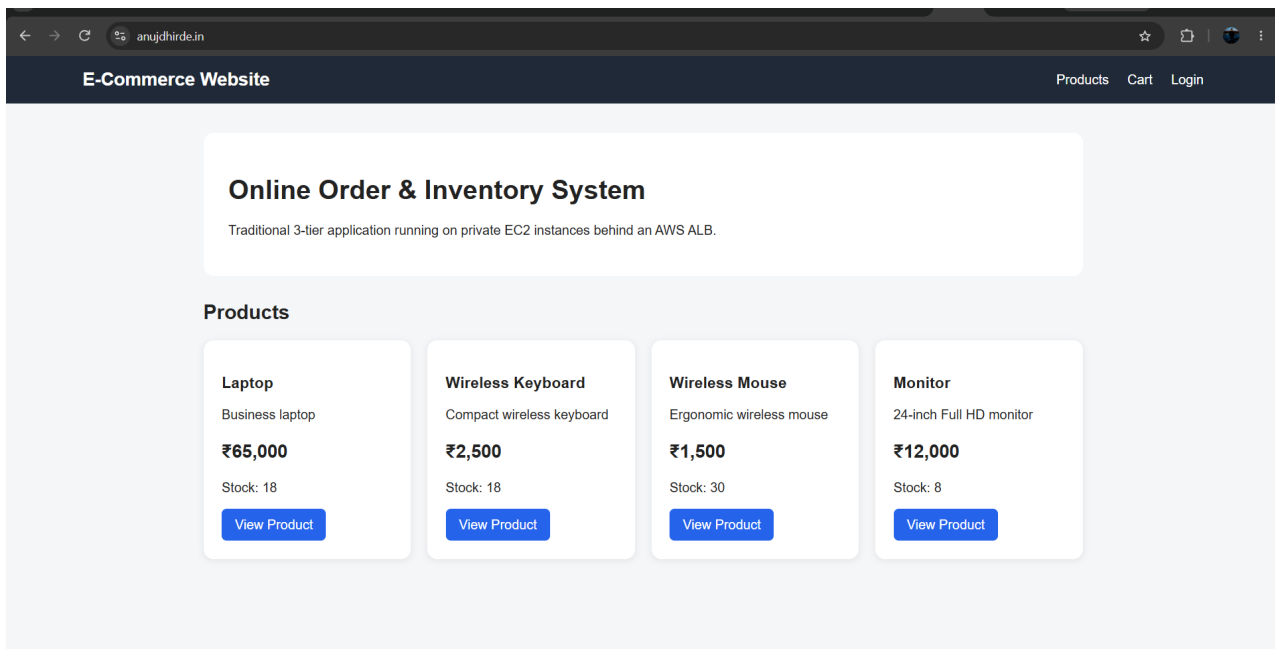


Figure 29 — User login.

10.3 Shopping Cart

Authenticated users can add products to the cart and review quantities, prices, and subtotals.

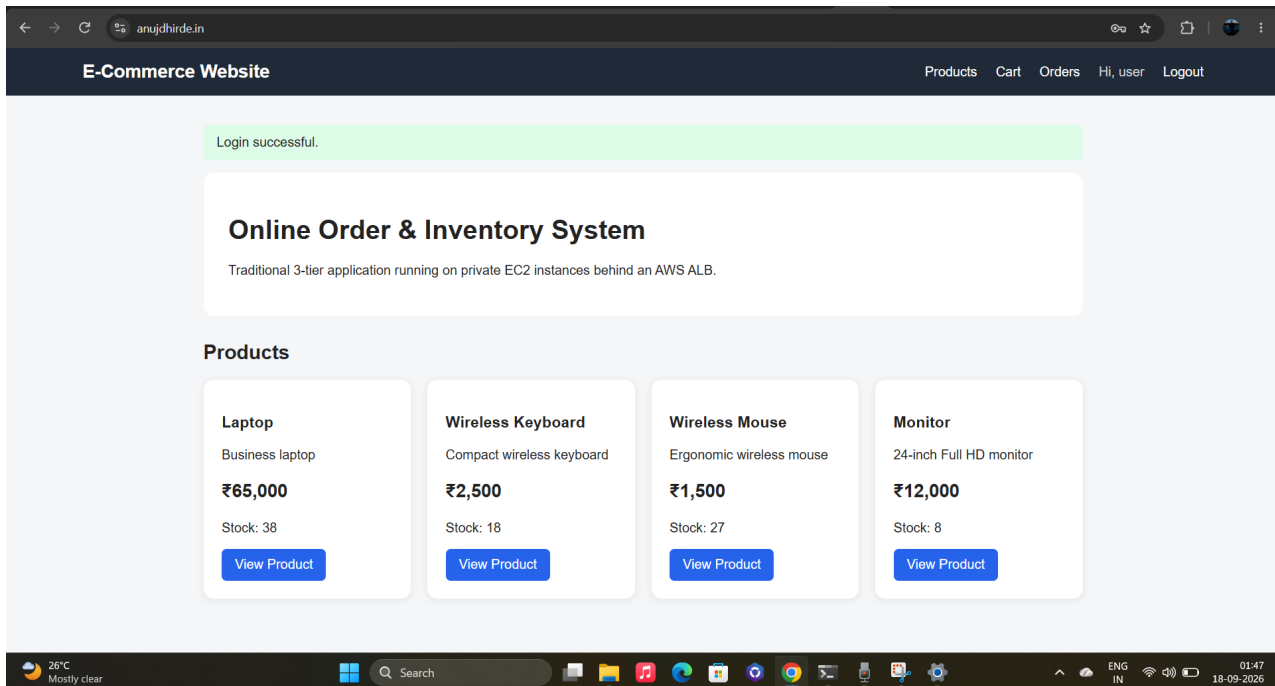


Figure 30 — Shopping cart.

10.4 Order Processing

The checkout workflow creates order data and updates inventory in PostgreSQL.

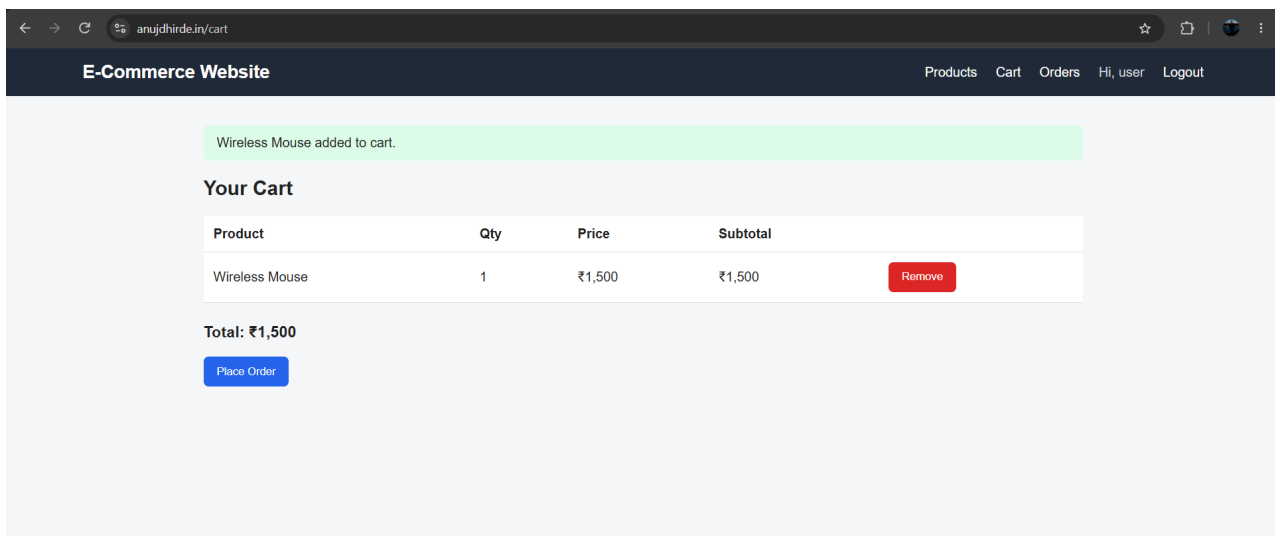


Figure 31 — Order confirmation and history.

10.5 Administrative Inventory Management

The administrative interface provides inventory visibility and allows stock values to be updated.

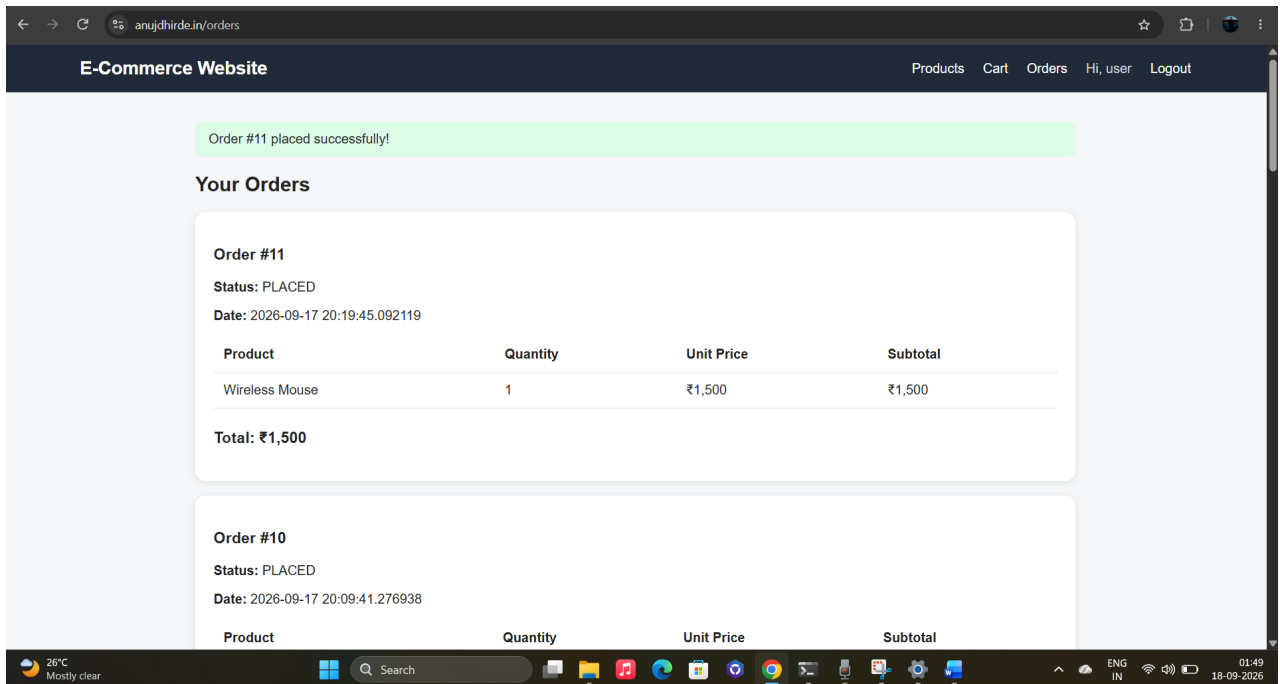


Figure 32 — Admin dashboard.

11. End-to-End Request Flow

User Browser → Route 53 → ALB / HTTPS → EC2 Application Tier → RDS PostgreSQL

EC2 → S3 Gateway VPC Endpoint → Amazon S3

Private EC2 → NAT Gateway → Internet Gateway → Internet

This separation keeps the application and database resources away from direct inbound internet exposure while providing the connectivity required for application delivery and operation.

12. Security Design

- Network isolation: application and database resources are separated from the public presentation tier.
- Controlled ingress: the ALB is the internet-facing entry point.
- HTTPS encryption: ACM provides the certificate used by the ALB HTTPS listener.
- Layered access control: traffic between tiers is controlled through security groups.
- Private S3 access: EC2 accesses S3 through a Gateway VPC Endpoint.
- IAM authorization: an IAM role grants EC2 access to the S3 bucket.
- Database isolation: PostgreSQL is deployed in the private database layer.

13. High Availability and Scalability

The application tier is distributed across two Availability Zones, with an EC2 instance in each application subnet. The ALB spans the two public subnets and distributes traffic to the registered application targets.

This design reduces dependence on a single application server or Availability Zone and provides a foundation for horizontal scaling by adding additional instances behind the target group.

Accuracy note: The supplied architecture diagram labels the EC2 layer as “Auto Scaling,” but the implementation notes document two EC2 instances rather than creation of an Auto Scaling Group. This report therefore documents the verified deployment as two EC2 instances behind the ALB.

14. Technologies Used

Cloud: AWS

Networking: Amazon VPC, Subnets, Route Tables, Internet Gateway, NAT Gateway, VPC Endpoint

Compute: Amazon EC2

Load Balancing: Application Load Balancer, Target Group

Database: Amazon RDS for PostgreSQL

Storage: Amazon S3

Security: IAM, Security Groups, AWS Certificate Manager

DNS: Amazon Route 53

Application: Python, Flask, PostgreSQL

Environment: Python virtual environment

15. Project Outcome

The project demonstrates hands-on implementation of a traditional AWS three-tier application, including network segmentation, multi-AZ application deployment, load balancing, private database connectivity, S3-based application deployment, IAM authorization, HTTPS termination, and database-backed e-commerce functionality.

The completed application supports product browsing, authentication, shopping-cart operations, order processing, inventory management, and administrative stock updates.

Thank You